

An Adversarial, Differential, and Reproducible Verification of *Unchessed-UCI-Engine* Against FIDE Chess Rules

Manus AI

Independent Verification Report

September 2026

Audited commit: 9c4f1f8e82dca8e4c729f275e8c15cb0e87e7b74

Abstract—This paper reports an adversarial verification of the open-source *Unchessed-UCI-Engine* at commit 9c4f1f8e. The study combines source inspection, the repository’s Rust and Python test suites, canonical perft references, an independently maintained chess-rules oracle, deterministic reachable-position stress testing, malformed-FEN probes, and targeted FIDE terminal-state tests. The engine compiled in debug and release profiles; 123 default Rust tests, 6 ignored Rust tests, and 385 Python tests passed, with 22 Python tests skipped. A branch-local integration test matched all six canonical perft positions through the repository’s published depths, including Kiwipete at depth five. An independent `python-chess` oracle matched 24 tractable canonical counts through depth four. In a 1,022-case UCI run, 1,011 of 1,011 non-terminal valid positions produced moves legal under the oracle, including 1,000 deterministic reachable positions. These results support move-generation soundness within the tested scope, but do not establish complete FIDE compliance. Three dead positions returned moves instead of an immediate draw, and rejected FEN commands left the previous position active while still producing a normal `bestmove`. Clock, touch-move, scoresheet, arbiter, venue, and tournament rules were correctly treated as outside ordinary UCI-engine observability. The paper therefore concludes partial confirmation, not a blanket claim of correctness.

Index Terms—Chess engine verification, FIDE Laws of Chess, perft, differential testing, UCI, adversarial testing, dead position, FEN validation.

I. INTRODUCTION

A chess engine can be strong while still violating the rules of chess. Search quality and evaluation accuracy do not imply correct move generation, state transitions, terminal adjudication, or protocol behavior. This distinction is especially important when a repository describes a recent checkmate-versus-halfmove correction but provides no single claim that can be accepted without independently reproducing the behavior.

This work audits *Unchessed-UCI-Engine* as an implementation artifact rather than as an advocacy exercise. The target commit, environment, commands, input positions, output transcripts, and generated evidence are preserved on the branch `manus/research-facilities`. The study asks four narrow questions: (i) does the current workspace build and pass its declared tests; (ii) does its move generator agree with canonical perft references; (iii) does its UCI executable return oracle-legal moves over targeted and reachable positions; and

(iv) does its observed terminal and input-validation behavior satisfy the relevant FIDE requirements?

The paper uses the FIDE Laws effective from 1 January 2023 as the primary rules authority [1]. The Laws explicitly cover over-the-board play and include both basic and competitive rules. Therefore, a rigorous result must distinguish rules observable through a position/search interface from rules requiring physical pieces, clocks, scoresheets, or an arbiter. Treating the latter as silently passed would inflate the evidence.

II. THREAT MODEL AND VERIFICATION CRITERIA

The threat model includes pseudo-legal moves that expose the moving side’s king, incorrect slider blocking, castling through attacked squares, stale en-passant state, promotion errors, repetition-key errors, halfmove-boundary errors, incorrect mate precedence, malformed-position crashes, and protocol fallbacks that return an answer for an input that was rejected. The study also considers false confidence caused by unit tests that do not exercise the released executable.

The evidence ladder is intentionally conservative. Source inspection establishes what current code says, but not what it does. An automated test establishes only the executed assertion. Runtime execution establishes behavior of the current binary on captured inputs. No finite corpus proves all reachable chess positions. The verdict labels used here are *confirmed*, *refuted*, *partially confirmed*, and *could not verify*.

For move legality, a returned UCI move was accepted only if it belonged to the independent oracle’s legal-move set for the exact FEN. For terminal behavior, a position was evaluated with the oracle’s game-status semantics and compared with the engine’s `bestmove` behavior. For perft, the expected counts were taken from the established Perft Results reference, whose purpose is debugging move generation and which includes the initial position, Kiwipete, and positions targeting captures, checks, castling, en passant, and promotions [2].

III. AUDITED ARTIFACT AND ENVIRONMENT

The repository was cloned from `Amoguslittleahhh/Unchessed-UCI-Engine`. The audited main-branch commit was 9c4f1f8e82dca8e4c729f275e8c15cb0e87e7b74, with commit message “Fix checkmate scored as

a draw at halfmove clock 100.” A new branch, `manus/research-facilities`, was created before research artifacts were added. No implementation source was edited for the audit; one branch-local integration test was added under `unchessed-core/tests`.

The Rust workspace contains four members: `unchessed-core`, `unchessed-adapter`, `unchessed-reviewer`, and `unchessed-datagen`. The core exposes FEN parsing, board state, legal move generation, `perft`, SAN, search, and UCI-related code. The repository also contains Python analysis tools, tests, model artifacts, configuration, documentation, and large PGN archives. All files were inventoried; the PGN archive was not treated as a legality oracle because historical game data is not a complete specification of FIDE state semantics.

The initial system Cargo 1.75.0 could not parse the repository’s lockfile version 4. A current stable Rust toolchain, `rustc/cargo 1.98.1`, was installed and recorded. Python dependencies were installed from the repository’s `tools/requirements-dev.txt`, including `python-chess 1.11.2` and `pytest`. The release executable was approximately 1.1 MB after compilation.

IV. METHODS

A. FIDE Clause Matrix

A 54-row matrix maps Articles 1–12 and Appendices A–D to scenario classes. Engine-testable rows cover board representation, piece movement, king safety, castling, en passant, promotion, terminal positions, FEN legality, repetition, halfmove state, and algebraic notation. OTB-only rows cover clocks, touch-move, physical displacement, scoresheets, draw offers, arbiter powers, player conduct, rapid/blitz administration, and tournament scoring. They remain in the matrix as explicit non-testable scope rather than being counted as successful.

B. Repository Test Reproduction

The declared script `scripts/build-and-test.sh` was run with the current toolchain. It performs a workspace build, workspace tests, a start-position UCI smoke test, and a forced back-rank-mate smoke test. The six ignored Rust tests were run separately with `cargo test --workspace -- --ignored`. The complete Python test surface was executed with `python3 -m pytest -q tools`. An optimized release workspace build was also performed.

C. Canonical Perft Cross-Checking

The repository’s existing `perft` tests contain six canonical positions and expected leaf-node counts. A new integration test reasserted all those counts and added the online Kiwipete depth-five count of 193,690,690. The test was compiled and run against the current `unchessed-core` implementation. Independently, a pure-Python `python-chess` counter recomputed 24 tractable counts through depth four. The deeper Kiwipete count was deliberately left to the Rust implementation because naive Python expansion to 193 million nodes is not a credible execution method.

TABLE I
REPRODUCED TEST AND BUILD RESULTS

Surface	Pass	Fail/skip	Interpretation
Rust default	123	0/6 ignored	Automated internal evidence
Rust ignored	6	0/0	Benchmark/deep checks completed
Python tools	385	0/22 skipped	Repository Python surface
Release build	–	0	Optimized workspace compiled
UCI smoke	2	0	Start position and forced mate

D. Differential UCI Stress Test

The UCI harness sent one persistent protocol session containing 16 fixed valid positions, 1,000 deterministic reachable positions, and 6 malformed-FEN inputs. Reachable positions were generated from the standard initial position using seed 20260904 and up to 60 legal plies. Each non-terminal returned move was checked against `python-chess.Board.legal_moves`. Fixed positions included castling rights, en-passant state, promotions, pins, king attacks, stalemate, dead material, and halfmove boundaries. Malformed cases included missing kings, two kings of one color, a pawn on the back rank, invalid castling rights, and an occupied en-passant square.

E. Targeted Terminal and Protocol Probes

The engine was tested at halfmove clocks 100 and 150 on both a forced mate and a valid non-check position. FIDE Article 9.6.2 gives checkmate precedence over the 75-move automatic draw [1]. Individual processes were used for dead positions and malformed FENs to prevent batch-output association errors. The exact output and parser messages were retained.

V. RESULTS

A. Build and Existing Tests

Table I summarizes the observed current runs. The default Rust suite reported 123 passed, zero failed, and six ignored. The ignored suite reported six passed. The Python suite reported 385 passed, 22 skipped, and 347 subtests passed. The release build and the declared UCI smoke checks completed successfully.

B. Perft and Oracle Results

The branch-local canonical integration test passed as one test and evaluated all six positions across their embedded depths, including the start position through depth five, Kiwipete through depth five, and positions 3–6 through their repository-defined depths. The independent Python oracle passed all 24 tractable count checks. These results provide Level 4 evidence for the move-generation implementation and independent reference agreement, but they do not demonstrate arbitrary-position completeness.

The expanded UCI run contained 1,022 cases. It reported 1,016 valid positions and six malformed inputs. Of the valid set, 1,011 non-terminal positions returned oracle-legal moves; no non-terminal legal-bestmove failure was observed. Five

TABLE II
EXPANDED UCI DIFFERENTIAL RUN

Group	Count	Observation
Fixed valid positions	16	Special-move, king-safety, terminal, and clock cases
Reachable valid positions	1,000	Deterministic seed; up to 60 legal plies
Non-terminal legal returns	1,011/1,011	All accepted by independent oracle
Malformed FEN inputs	6	Parser behavior retained verbatim
Dead-position probes	3	Ordinary move returned; FIDE gap
Stalemate probe	1	<code>bestmove 0000</code>

valid positions were terminal under the oracle and were analyzed separately rather than incorrectly classified as legal-move failures.

C. Mate and Halfmove Precedence

For the forced back-rank mate, the executable returned `bestmove ala8` and reported mate in one at both halfmove clocks 100 and 150. This agrees with the FIDE rule that a checkmate resulting from the last move takes precedence over the 75-move automatic draw. Source inspection also found an explicit guard in `unchessed-core/src/search.rs:519--530`. This is confirmed at runtime for the tested construction, not generalized to every mating position.

VI. ADVERSARIAL FINDINGS

A. Dead Positions Return Moves

FIDE Article 5.2.2 states that a position is drawn when neither player can checkmate the opponent by any series of legal moves [1]. The current executable returned `bestmove alb2` for king versus king, king and bishop versus king, and king and knight versus king. Source inspection found halfmove and repetition draw handling but no corresponding dead-material adjudication path. This is a Level 5 refutation of the narrower claim that these tested dead positions are automatically adjudicated as draws. It does not establish that every conceivable dead position is mishandled.

B. Rejected FEN Leaves Stale State Active

The UCI loop prints “could not parse” when `parse_position` returns `None`, while the outer loop retains the previous `Game`. The parser uses `fen::parse(...).ok()`?. In isolated probes, a missing-king FEN and a two-white-king FEN produced the parser error followed by `bestmove d2d4`, the move associated with the prior start position. Invalid castling rights and an occupied en-passant square showed the same pattern. This is a confirmed Level 5 protocol hazard: a client that ignores informational output can attribute a stale-state move to a rejected position.

A pawn on the back rank was accepted and the engine returned a promotion move. FIDE describes promotion as

mandatory upon reaching the furthest rank and defines an illegal position as one that cannot be reached by legal moves [1]. Whether accepting a promotion-ready FEN is a deliberate input-policy choice is unresolved; the implementation does not document the distinction sufficiently for this audit to classify it as a definite defect.

C. Fifty- and Seventy-Five-Move State

On valid non-check positions with halfmove clocks 100 and 150, the engine continued with a legal move rather than exposing a result or claim state. FIDE Articles 9.3 and 9.6 distinguish a claim after 50 moves from an automatic draw after 75 moves [1]. A UCI engine may delegate claims to a GUI, but the tested interface did not expose a documented claim/result response. Complete game-result compliance for these rules therefore remains unverified.

VII. DISCUSSION

The evidence supports a useful but bounded conclusion. The move generator is consistent with canonical perft references and with an independent legal-move oracle over the tested corpus. The repository’s internal tests are broad and execute successfully under a current toolchain. The observed mate/halfmove guard is a positive targeted result. These facts justify confidence in the tested implementation surfaces, not a claim that the entire FIDE Laws are implemented.

The dead-position result is particularly important because it is not a search-strength issue. A move can be legal under Articles 3.1–3.9 while the game must already be drawn under Article 5.2.2. Similarly, parser logging is not equivalent to protocol-safe rejection. A caller needs a way to know that the position was not installed before trusting a later `bestmove`.

The online authorities used here have different roles. FIDE is the normative rules source. The Chess Programming Wiki provides canonical perft values intended for debugging [2]. The Shredder UCI overview establishes the engine/GUI protocol context [3]. The `python-chess` core documentation identifies the independent Board abstraction and legal-move, status, FEN, castling, en-passant, and halfmove APIs used by the oracle [4]. No search-result snippet was treated as evidence; the pages were opened and captured locally.

VIII. THREATS TO VALIDITY AND NOT VERIFIED

The reachable-position sample is deterministic but finite; it is not exhaustive over the game graph. A `bestmove` legality check does not prove that every legal move is generated, only that the selected move was legal. Perft provides stronger enumeration for its tested depths but remains bounded by selected positions and depths. The UCI protocol probes used shallow searches and do not characterize all time controls, threads, options, model files, memory sizes, or long-running behavior.

The audit did not test physical one-hand movement, touch-move obligations, clock operation, flag falls, default time, scoresheet recording, draw offers, arbiter intervention, player conduct, venue or device rules, penalties, appeals, rapid/blitz

administration, or tournament points. These are not observable through ordinary UCI position/search commands. They are listed in the clause matrix as out of scope, not as successful.

The pure-Python deep perft attempt was stopped because expansion to the 193,690,690-node Kiwipete count was not tractable within the execution budget. This is a method limitation, not a failed chess result; the deeper count was still tested by the Rust integration test. The initial pipeline wrapper also produced a shell exit-status artifact despite a log ending in “OK: build + tests + smoke passed”; direct commands were rerun with explicit exit capture, and those reruns returned zero.

IX. CONCLUSION AND FOLLOW-UP

This study partially confirms the implementation’s move-generation and build claims at Levels 4–5 within the measured scope. It refutes complete automatic dead-position handling for three tested classes and confirms a stale-state hazard after malformed FEN rejection. It cannot verify the full FIDE Laws because substantial portions are competition procedures rather than engine algorithms.

A remediation should add explicit dead-position adjudication or a documented adjudication layer, with acceptance tests for dead and non-dead minor-piece positions. Invalid `position fen` should either fail closed with structured protocol state or document that subsequent search is intentionally against the prior position. Repetition, 50-move claims, 75-move automatic draws, castling-right identity, en-passant identity, and mate precedence should be exposed through explicit result tests rather than inferred from search scores.

All source, matrix, scripts, raw transcripts, results, and this paper’s LaTeX source are committed on `manus/research-facilities`. The strongest defensible conclusion is therefore: **the engine passes substantial move-generation and repository test evidence, but full FIDE compliance is not established and specific runtime gaps are reproducible.**

REFERENCES

- [1] FIDE, “FIDE Laws of Chess taking effect from 1 January 2023,” FIDE Handbook, approved 2022, applied Jan. 1, 2023. [Online]. Available: <https://handbook.fide.com/chapter/e012023>
- [2] Chess Programming Wiki, “Perft Results.” [Online]. Available: https://chessprogramming.org/Perft_Results
- [3] Shredder Chess, “Universal Chess Interface (UCI).” [Online]. Available: <https://www.shredderchess.com/chess-features/uci-universal-chess-interface.html>
- [4] Niklas Fiekas, “python-chess Core documentation,” version 1.11.2. [Online]. Available: <https://python-chess.readthedocs.io/en/latest/core.html>